

AGENTES IA

Master en Agentes RAG & Inteligencia Artificial Autónomos

Aprende a construir agentes inteligentes autónomos con LangChain, bases vectoriales (Pinecone) y modelos LLM conectados a APIs empresariales.

[Nivel: Avanzado]

[Duración: 18hs]

[12 Módulos]

[Tech: Python, LangChain, Pinecone, FastAPI]

#> Módulo 1: Arquitectura de Agentes y LLMs

Bienvenido al nivel avanzado de la Inteligencia Artificial. Hasta ahora, probablemente hayas usado ChatGPT como un oráculo: le haces una pregunta y te da una respuesta. Sin embargo, en entornos empresariales, eso no es suficiente. Queremos que la IA actúe por sí misma.

¿Qué es un Agente Autónomo?

Un agente es un sistema donde el modelo de lenguaje grande (LLM) funciona como el 'cerebro' o motor de razonamiento de una aplicación. El LLM decide en qué orden ejecutar las tareas, qué herramientas usar (como buscar en Google o consultar una base de datos) y cuándo ha terminado la tarea.

Preparando tu entorno

Para comenzar, necesitas una cuenta en OpenAI (platform.openai.com) para obtener una API Key, ya que usaremos GPT-4o como nuestro motor de razonamiento. Además, instalaremos el ecosistema LangChain.

```
pip install langchain langchain-openai python-dotenv

# Crea un archivo llamado .env en tu proyecto y añade:
OPENAI_API_KEY="sk-tu-clave-secreta"
LANGCHAIN_API_KEY="lsv2_tu-clave"
```

El uso del archivo ``.env`` es una práctica estándar de ciberseguridad para evitar subir contraseñas a GitHub. Usaremos la librería `python-dotenv` para cargar estas variables a nuestro script automáticamente.

#> Módulo 2: Profundizando en LangChain

LangChain es el framework estándar de la industria para construir aplicaciones impulsadas por LLMs. Su concepto principal es la 'Cadena' (Chain), que permite secuenciar componentes.

Componentes Principales (LCEL)

El Lenguaje de Expresiones de LangChain (LCEL) nos permite usar el símbolo '|' (pipe) para conectar un Prompt, un Modelo y un Parser, de forma similar a como funcionan las terminales de Linux.

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv() # Carga tu API KEY

# 1. Definimos el modelo
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)

# 2. Definimos la plantilla del prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", "Eres un asistente sarcástico pero útil."),
    ("user", "Explicame qué es {tema}")
])

# 3. Definimos el Output Parser
parser = StrOutputParser()

# Creamos la Cadena
cadena = prompt | llm | parser

# Invocamos la cadena
resultado = cadena.invoke({"tema": "el Machine Learning"})
print(resultado)
```

Explicación:

- Temperature (0.2): Controla la creatividad. Un valor bajo (0) hace que el modelo sea más determinista y analítico. Un valor alto (1) lo hace más creativo.

#> Módulo 3: Embeddings y Representación Semántica

Los LLMs no 'leen' texto como los humanos. Operan con matemáticas. Para que la IA pueda buscar documentos en base a su 'significado' (búsqueda semántica), debemos convertir el texto en vectores.

El Espacio Vectorial

Un Embedding es un vector matemático (una lista de miles de números) que representa el significado de un texto. Textos con significado similar estarán 'cerca' matemáticamente en este espacio multidimensional.

```
from langchain_openai import OpenAIEmbeddings

# Inicializamos el modelo generador de embeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

texto_1 = "El perro juega en el parque"
texto_2 = "Un canino se divierte en la plaza"

# Convertimos el texto a números
vector_1 = embeddings.embed_query(texto_1)
vector_2 = embeddings.embed_query(texto_2)

print(f"Dimensiones del vector: {len(vector_1)}")
print("Primeros 5 valores:", vector_1[:5])
```

Aunque las palabras en 'texto_1' y 'texto_2' son diferentes, sus vectores resultantes serán matemáticamente muy similares, permitiendo a la IA saber que hablan de lo mismo.

#> Módulo 4: Bases de Datos Vectoriales (Pinecone)

Una base de datos SQL tradicional (como MySQL) no sirve para guardar vectores de 1500 dimensiones y buscar similitudes rápidamente. Para ello usamos Pinecone, una Base de Datos Vectorial especializada en milisegundos.

Conectando e Insertando Datos

Primero, necesitas instalar el SDK: ``pip install pinecone-client langchain-pinecone`` y configurar ``PINECONE_API_KEY`` en tu ``.env``.

```
import os
from pinecone import Pinecone, ServerlessSpec
from langchain_pinecone import PineconeVectorStore

pc = Pinecone(api_key=os.getenv("PINECONE_API_KEY"))
index_name = "curso-rag"

# Crear el índice si no existe
if index_name not in pc.list_indexes().names():
    pc.create_index(
        name=index_name,
        dimension=1536, # Dimensión del modelo de OpenAI
        metric="cosine",
        spec=ServerlessSpec(cloud="aws", region="us-east-1")
    )

# Insertar documentos reales
textos = [
    "Las políticas de reembolso de la empresa exigen 30 días.",
    "El horario de soporte técnico es de 9am a 6pm EST."
]

vectorstore = PineconeVectorStore.from_texts(
    textos,
    embeddings,
    index_name=index_name
)
```

Explicación:

#> Módulo 5: Arquitectura RAG Completa

RAG (Retrieval-Augmented Generation) resuelve el problema de las 'alucinaciones' del LLM. En lugar de depender de lo que el modelo memorizó durante su entrenamiento, le damos el documento exacto ANTES de que responda.

Flujo RAG de Extremo a Extremo

1. Recuperador (Retriever): Busca los 3 documentos más relevantes a la pregunta del usuario en Pinecone.
2. Inyección de Contexto: Pega esos documentos en el Prompt.
3. Generación: El LLM lee el prompt con el contexto y responde.

```
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain.chains import create_retrieval_chain

# El retriever buscará en Pinecone
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

prompt = ChatPromptTemplate.from_template("""
Responde a la pregunta basándote ÚNICAMENTE en el siguiente contexto:

<contexto>
{context}
</contexto>

Pregunta: {input}
""")

# Creamos la cadena que inyecta documentos
document_chain = create_stuff_documents_chain(llm, prompt)

# Unimos el recuperador con la cadena
rag_chain = create_retrieval_chain(retriever, document_chain)

respuesta = rag_chain.invoke({"input": "¿Cuál es el horario de soporte?"})
print(respuesta["answer"])
```

Gracias a esto, el LLM responderá 'De 9am a 6pm EST' basándose en los datos corporativos seguros.

#> Módulo 6: Tools y Conexión a APIs

El RAG nos permite leer documentos estáticos. Pero un Agente IA real necesita escribir correos, consultar bases de datos SQL, buscar el clima o revisar precios de acciones en vivo. Esto se logra mediante 'Herramientas' (Tools).

Definiendo Herramientas para el Modelo

Una herramienta en LangChain es una función de Python decorada, que obligatoriamente requiere docstrings claros, porque el LLM los lee para entender para qué sirve la herramienta.

```
from langchain_core.tools import tool
import requests

@tool
def consultar_clima(ciudad: str) -> str:
    """Utiliza esta herramienta EXCLUSIVAMENTE para obtener el clima actual de una ciudad especificada."""
    # Simulación de un llamado a API REST:
    # respuesta = requests.get(f"https://api.clima.com/?q={ciudad}")
    # return respuesta.json()['temperatura']
    return f"El clima en {ciudad} es 24°C, parcialmente nublado."

@tool
def calculadora(expresion: str) -> str:
    """Resuelve expresiones matemáticas complejas."""
    return str(eval(expresion))

lista_herramientas = [consultar_clima, calculadora]
```

Ahora tenemos funciones de Python que el modelo puede decidir usar por su cuenta.

#> Módulo 7: El Framework ReAct (Reasoning and Acting)

ReAct es el patrón de diseño donde la IA piensa paso a paso. Se hace una pregunta, la IA 'Piensa' qué herramienta necesita usar, 'Actúa' invocando la herramienta, y 'Observa' el resultado. Repite el ciclo hasta tener la respuesta final.

Implementación del Agente Autónomo

Combinaremos nuestras herramientas con el modelo.

```
from langchain.agents import create_tool_calling_agent, AgentExecutor

# Obligamos al modelo a usar las herramientas
llm_con_tools = llm.bind_tools(lista_herramientas)

prompt_agente = ChatPromptTemplate.from_messages([
    ("system", "Eres un asistente AI autónomo poderoso."),
    ("user", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])

agente = create_tool_calling_agent(llm, lista_herramientas, prompt_agente)

# El Executor es el motor que ejecuta el bucle while infinito de ReAct
ejecutor = AgentExecutor(agent=agente, tools=lista_herramientas, verbose=True)

respuesta = ejecutor.invoke({"input": "¿Cuál es la raíz cuadrada de 144, y cómo está el clima en París?"})
print(respuesta['output'])
```

Explicación:

- `verbose=True` es esencial para desarrolladores: te mostrará en la terminal cada pensamiento y acción que toma el modelo en tiempo real.
- `agent\_scratchpad` es la memoria a corto plazo donde el agente guarda los pasos que ya intentó.

#> Módulo 8: Memoria y Contexto a Largo Plazo

Un agente sin memoria sufre de amnesia con cada mensaje nuevo. En sistemas de IA, la memoria se logra enviando todo el historial de chat anterior en cada nuevo llamado al modelo.

Tipos de Memoria

Existen memorias de Buffer (guardan todo), y memorias de Resumen (usan otro LLM pequeño para resumir charlas viejas y ahorrar tokens). Usaremos BufferMemory para un Agente.

```
from langchain.memory import ConversationBufferMemory

# Creamos una memoria que guarda mensajes
memoria = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True
)

# Modificamos el prompt para incluir el historial:
prompt_con_memoria = ChatPromptTemplate.from_messages([
    ("system", "Eres un asistente AI."),
    ("placeholder", "{chat_history}"), # Inyección dinámica de memoria
    ("user", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])

# Reconstruimos el agente inyectando la memoria al executor...
ejecutor_memorioso = AgentExecutor(
    agent=agente,
    tools=lista_herramientas,
    memory=memoria,
    verbose=True
)
```

#> Módulo 9: Sistemas Multi-Agente con LangGraph

Los casos de uso corporativos (como programar código, revisar vulnerabilidades y compilar) son demasiado complejos para un solo agente. LangGraph permite crear flujos de trabajo (Grafos) con múltiples agentes especializados.

Flujo de Nodos

En LangGraph, defines un 'Estado' (un diccionario). Cada nodo (Agente A, Agente B) toma el estado, lo modifica, y se lo pasa al siguiente nodo. Se define la lógica de enrutamiento con Edge Conditions.

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
import operator

class EstadoAgente(TypedDict):
    mensajes: Annotated[list, operator.add]

def nodo_investigador(estado):
    # Lógica del agente investigador...
    return {"mensajes": ["Encontré esta información."]}

def nodo_redactor(estado):
    # Lógica del agente redactor...
    return {"mensajes": ["Escribiendo reporte..."]}

grafo = StateGraph(EstadoAgente)
grafo.add_node("Investigador", nodo_investigador)
grafo.add_node("Redactor", nodo_redactor)

# Flujo
grafo.add_edge(START, "Investigador")
grafo.add_edge("Investigador", "Redactor")
grafo.add_edge("Redactor", END)

app = grafo.compile()
```

Este grafo determinista nos permite construir procesos empresariales

#> Módulo 10: Desarrollo de Backend con FastAPI

Un script en tu computadora no puede ser utilizado por miles de clientes. Necesitamos crear un Backend robusto (API REST) que sirva tu Agente a cualquier front-end (React, Vue, Android).

Exponiendo el Agente al mundo

Instalamos FastAPI y el servidor Uvicorn: ``pip install fastapi uvicorn pydantic``

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Servidor de Agentes IA")

# Modelo de validación de datos de entrada
class PeticionUsuario(BaseModel):
    session_id: str
    mensaje: str

@app.post("/api/v1/chat")
async def chatear(peticion: PeticionUsuario):
    try:
        # Invocamos al agente que creamos en módulos anteriores
        # En un sistema real, recuperaríamos la memoria basada en session_id
        respuesta = ejecutor.invoke({"input": peticion.mensaje})
        return {
            "status": "success",
            "respuesta_ia": respuesta["output"]
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# Para correr el servidor, usa en tu terminal:
# uvicorn archivo:app --reload
```

#> Módulo 11: Despliegue y Monitorización

Lanzar IA a producción es peligroso si no sabes qué está haciendo el modelo internamente. LangSmith es el dashboard oficial de LangChain para trazabilidad.

LangSmith Tracing

Añadiendo 3 variables a tu `.env`, todo el tráfico, uso de tokens, latencias y errores de tus agentes se guardarán en un panel en vivo en la nube.

```
LANGCHAIN_TRACING_V2="true"
LANGCHAIN_ENDPOINT="https://api.smith.langchain.com"
LANGCHAIN_API_KEY="lsv2_tu_clave_smith"
LANGCHAIN_PROJECT="agente_produccion_v1"
```

Una vez validado el agente, usamos Docker para empaquetar el código de FastAPI, sus dependencias de Python y variables de entorno en un contenedor aislado, listo para AWS o Google Cloud.

#> Módulo 12: Proyecto Final (Agente Empresarial)

El examen final. Combinarás TODO lo aprendido. Vas a desarrollar un 'Representante de Ventas Autónomo'.

Arquitectura Requerida

1. Embeddings & RAG: Ingestarás 50 PDFs de catálogos de productos a Pinecone.
2. Tools de Base de Datos: Desarrollarás una herramienta SQLTool para que el agente consulte el stock en vivo de una base MySQL.
3. Framework ReAct: El agente debe deducir el producto que quiere el cliente, buscar detalles en RAG, verificar stock con SQLTool, y calcular precio con descuento usando una MathTool.
4. Backend: Todo debe estar montado en FastAPI con rutas seguras.

¡Cuando completes esto, oficialmente serás un Arquitecto de Inteligencia Artificial! El mercado corporativo está buscando este nivel de expertise a nivel global.

MASTER EN AGENTES IA & RAG

Certificación otorgada a:

[ARCHITECT_NAME]

Por haber dominado el diseño, desarrollo y despliegue de Sistemas Multi-Agente Autónomos, integrando arquitecturas RAG, Bases Vectoriales y desarrollo Backend.

TIMESTAMP: 2026-08-12 // AI_MASTER_NODE_SIGNATURE